

```
```sh
sh kops.sh
```
```

```
```sh
export KOPS_STATE_STORE=s3://reyaz-kops-testbkt143.k8s.local
```
```

```
```sh
kops validate cluster --wait 10m
```
```

```
```sh
kubectl get nodes
```
```

```
```bash
cat <<EOF > selector.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: lb-deployment
 labels:
 app: bank
spec:
 replicas: 2
 selector:
 matchLabels:
 app: bank
 template:
 metadata:
 labels:
 app: bank
 spec:
 nodeSelector:
 node-name: lb-node
 containers:
 - name: cont1
 image: reyadocker/internetbankingrepo:latest
EOF
```
```

```
```bash
vi nodesselector.yml
```

```
kubectl apply -f nodesselector.yml
```

```
kubectl get pods -o wide
```
```

```
```bash
kubectl get events
```
```

```
```bash
kubectl logs lb-deployment-7784c9bfb5-fv5qd
```

```
kubectl get events
```
```

```
```bash
kubectl describe pod lb-deployment-7784c9bfb5-fv5qd
```
```

```
```bash
kubectl get nodes
```

```
kubectl edit node i-004f9b5037e50bcfd
```
```

Edit node-name: ib-node like the following file:

Or use this command :

```
```bash
kubectl label node i-004f9b5037e50bcfd node-name=lb-node --overwrite
```
```

```
```
kubectl delete -f nodeselector. yml
Kibectl get pods
```
```

```
```
vi preferred.yml
```bash
cat <<EOF > preferred.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lb-deployment
  labels:
    app: bank
spec:
  replicas: 1
  selector:
```

```
  matchLabels:
    app: bank
template:
  metadata:
    labels:
      app: bank
  spec:
    affinity:
      nodeAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 1
            preference:
              matchExpressions:
                - key: node-name
                  operator: In
                  values:
                    - mb-node
        containers:
          - name: nginx
            image: nginx:1.14.2
            ports:
              - containerPort: 80
```

EOF

```

```
kubectl get nodes
```

```

```bash

```
kubectl apply -f preferred.yaml
```

```
kubectl get pods -o wide
```

```

```bash

```
kubectl delete -f preferred.yaml
```

```

```bash

```
cat <<EOF > required.yml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
 name: lb-deployment
```

```
 labels:
```

```
 app: bank
```

```
spec:
```

```
 replicas: 1
```

```
 selector:
```

```
 matchLabels:
```

```

 app: bank
template:
 metadata:
 labels:
 app: bank
 spec:
 affinity:
 nodeAffinity:
 requiredDuringSchedulingIgnoredDuringExecution:
 nodeSelectorTerms:
 - matchExpressions:
 - key: node-name
 operator: In
 values:
 - mb-node
 containers:
 - name: nginx
 image: nginx:1.14.2
 ports:
 - containerPort: 80

```

EOF

```

```bash

kubectl create -f required.yml

kubectl get pods -o wide

kubectl delete -f required.yml

clear

```

Why Master Node not schedule any job on itself the ans is tent keep as no -schedule check here by pasting masster id

```bash

kubectl get nodes

kubectl edit node i-02e9e62e21f186d3f

```

List all nodes

[root@kops ~]# kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
i-004f9b5037e50bcfd	Ready	node	28m	v1.25.16
i-02e9e62e21f186d3f	Ready	control-plane	29m	v1.25.16
i-0e20e80d62aba4c2c	Ready	node	28m	v1.25.16

```
# Try to edit node (cancelled)
[root@kops ~]# kubectl edit node i-02e9e62e21f186d3f
Edit cancelled, no changes made.
```

```
# Taint GPU node to prevent scheduling regular pods
[root@kops ~]# kubectl taint nodes i-0e20e80d62aba4c2c hardware=gpu:NoSchedule
node/i-0e20e80d62aba4c2c tainted
```

```
...
```bash
vi taint.yml
```bash
cat <<EOF > taint.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ib-deployment
  labels:
    app: bank
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bank
  template:
    metadata:
      labels:
        app: bank
    spec:
      containers:
        - name: cont1
          image: reyadocker/internetbankingrepo:latest
EOF
...

```

```
kubectl create -f taint.yml
```

```
kubectl get pods -o wide
```

```
kubectl delete -f taint.yml
...
```

```
```bash
vi taint.yml
```bash
cat <<EOF > tolerance.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ib-deployment
  labels:
    app: bank
spec:
  replicas: 1
  selector:
    matchLabels:
      app: bank
  template:
    metadata:
      labels:
        app: bank
    spec:
      containers:
        - name: cont1
          image: reyadocker/internetbankingrepo:latest
      tolerations:
        - key: "hardware"
          operator: "Equal"
          value: "gpu"
          effect: "NoSchedule"
EOF
```
```

```
kubectl create -f tolerance.yml
```

```
kubectl get pods -o wide
```

```
kubectl delete -f tolerance.yml
```

```
```
```

```
```bash
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
helm version
```
```

Install ARGOCD using HELM

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
```

```
kubectl create namespace argocd
helm install argocd argo/argo-cd --namespace argocd
kubectl get all -n argocd
```

EXPOSE ARGOCD SERVER:

```
kubectl patch svc argocd-server -n argocd -p '{"spec":
{"type": "LoadBalancer"}}'
```

```
yum install jq -y
```

```
//export ARGOCD_SERVER='kubectl get svc argocd-server -n
argocd -o json | jq --raw-output
'.status.loadBalancer.ingress[0].hostname''
```

```
//echo $ARGOCD_SERVER
```

```
kubectl get svc argocd-server -n argocd -o json | jq
--raw-output '.status.loadBalancer.ingress[0].hostname'
```

The above command will provide load balancer URL to access
.GO CD

Direct command – get and decode password in one line

```
kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath="{.data.password}" | base64 -d
```

Store password in a variable

```
export ARGO_PWD='kubectl -n argocd get secret
argocd-initial-admin-secret -o jsonpath="{.data.password}" |
base64 -d'
```

Print the password

```
echo $ARGO_PWD
```

The above command to provide password to access argo cd

TO GET ARGO CD PASSWORD:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o  
jsonpath="{.data.password}" | base64 -d
```

```
export ARGO_PWD='kubectl -n argocd get secret  
argocd-initial-admin-secret -o jsonpath="{.data.password}" |  
base64 -d'
```

```
echo $ARGO_PWD
```

The above command to provide password to access argo cd
Verify pods were created automatically by ArgoCD
kubectl get po --> it created automatically from argocd

Get services – copy paste the elb on browser
kubectl get svc --> copy paste the elb on browser

Open ArgoCD load balancer

username: admin and password from above command

create app --> Application Name --> bankapp --> Project Name
--> default -->
Sync Policy --> Automatic --> Repository -->
<https://github.com/ReyazShaik/ar-deploy.git>
--> Path --> ./ --> Cluster URL --> Namespace --> default

kubectl get po --> it created automatically from argocd

kubectl get svc --> copy paste the elb on browser

now modify replica as 5 in GitHub deploy.yml , automatically
argocd will deploy